

PROJECT REPORT

Bookstore Management System

Domain: Software Engineering / Terminal Application

Technology Stack: Python 3, PrettyTable Library

Data Structure: In-Memory Key-Value Stores

2. Introduction

The Bookstore Management System is a console-based software application engineered to manage core retail bookstore operations, including inventory administration, customer sales transactions, stock tracking, and automated bill generation. Developed in Python 3, the system utilizes in-memory dictionaries coupled with the PrettyTable library to present clear, structured data grids directly inside the terminal interface.

3. Problem Statement

Small-to-medium retail bookshops often rely on paper registers or unwieldy spreadsheets for tracking inventory and sales. This practice frequently results in stock discrepancies, sluggish manual search times across extensive catalogs, and time-consuming invoice calculations. There is a need for a lightweight, standalone, digital application that facilitates instant multi-criteria catalog searching, real-time inventory adjustments, and itemized customer invoice generation.

4. Functional Requirements

- **Admin Catalog Insertion:** Enables administrators to insert new book entries with Title, Author, Publisher, Year, Price, and Stock.
- **Catalog Display:** Visualizes all catalog records in formatted tabular representations using PrettyTable.
- **Multi-Parametric Search:** Supports granular queries across 6 criteria (Title substring, Book ID, Publisher substring, Publication Year, Year Range, and Author).
- **Record Modification:** Permits in-place field updates for any attribute of an existing book ID.
- **Record Deletion:** Removes book entries permanently from active inventory by ID.
- **Customer Purchasing:** Captures customer profiles, manages multi-item cart purchases, validates stock levels, and deducts sold units in real time.
- **Billing & Invoicing:** Generates itemized order receipts detailing line items, quantities, prices, and total payable amounts.

5. Non-functional Requirements

- **Usability:** Text-driven menu options ensure intuitive navigation and rapid user adoption.
- **Data Presentation:** Single-border grid styling with aligned tabular column formatting prevents cluttered terminal outputs.
- **Portability:** Runs on any workstation supporting standard Python 3.x and the PrettyTable package without heavy external database engines.
- **Performance:** Direct dictionary key hashing ensures $O(1)$ average lookup performance for inventory and order retrieval.

6. System Architecture

The application utilizes a procedural, menu-driven control architecture. An execution loop in the main script dispatches control across three distinct sub-modules: Admin Panel, Purchase Engine, and Invoice/Billing Processor. Runtime data management is split across two central key-value collections: the Bookstore catalog dictionary and the orders dictionary.

7. Design Diagrams

- **Use Case Model:** Identifies two primary actors: 'Administrator' (executes CRUD catalog functions) and 'Customer/Cashier' (executes purchases and invoice queries).
- **Workflow Model:** Maps program flow starting at Main Menu (1-4) branching into Admin sub-menus (1-6) or Purchase/Billing loops.
- **Sequence Model:** Traces execution: User selects Book ID -> System queries Bookstore dict -> Checks stock against requested qty -> Prompts override if insufficient -> Updates stock and appends order to orders dict.
- **Class/Component Structure:** Defines schema Bookstore[book_id] = [title, author, publisher, pub_year, price, stock] and orders[order_id] = [[cust_name, addr, phone, email, book_id, title, price, qty, total]].
- **ER Diagram:** Not applicable because the system operates purely via in-memory Python structures rather than relational SQL storage tables.

8. Design Decisions & Rationale

- **In-Memory Hash Maps (Dictionaries):** Chosen to provide zero-configuration local deployment and lightning-fast read/update speeds without SQL setup overhead.
- **PrettyTable Library:** Incorporated PrettyTable to render enterprise-style ASCII borders, ensuring clean tabular presentation on any terminal window size.
- **Stock Guard Logic:** Prevents out-of-stock ordering by alerting users when requested quantities exceed current stock, allowing full remaining inventory claiming.

9. Implementation Details

The central inventory repository is constructed using an integer-keyed dictionary of lists:

```
Bookstore = {
    101: ["The Alchemist", "Paulo Coelho", "HarperCollins", 1988, 399, 25],
    102: ["Wings of Fire", "A.P.J. Abdul Kalam", "Universities Press", 1999, 350, 18],
    # ... additional records
}

# Stock Validation Logic:
if qty >= stock:
```

```

    print("Only " + str(stock) + " copies remaining. Order anyway? y/n")
    if input() == "y":
        qty = stock
        Bookstore[bookid][5] = 0
    else:
        Bookstore[bookid][5] -= qty

```

10. Screenshots / Results

Console Interface Output:

```

-----BOOKSTORE MANAGEMENT-----
                1. ADMIN PANEL
                2. PURCHASE
                3. INVOICE/BILLING
                4. EXIT
Enter your choice: 1

+-----+-----+-----+-----+-----+
+-----+-----+
| BOOK ID | TITLE                | AUTHOR                | PUBLISHER                | YEAR OF PUB.
| PRICE | STOCK |
+-----+-----+-----+-----+-----+
+-----+-----+
| 101     | The Alchemist         | Paulo Coelho          | HarperCollins           | 1988
| 399     | 25                    |                        |                          |
| 102     | Wings of Fire         | A.P.J. Abdul Kalam    | Universities Press      | 1999
| 350     | 18                    |                        |                          |
+-----+-----+-----+-----+-----+
+-----+-----+

```

11. Testing Approach

- **Admin Module Testing:** Validated CRUD menu branches, sub-menu returns, and data modifications on the Bookstore dictionary.
- **Search Verification:** Tested partial substring matching for authors and publishers as well as boundary range integer filters.
- **Stock Boundary & Transaction Testing:** Tested purchase requests exceeding stock to verify that stock clamp and deduction branches perform correctly.

12. Challenges Faced

- **Persistence Absence:** In-memory variables reset upon script termination, requiring all records to be stored transiently.
- **Inventory Synchronization:** Synchronizing cart arrays with active inventory balances during multi-item purchase sessions.

13. Learnings & Key Takeaways

- **Dictionary Modeling:** Gained deep experience modeling multi-relational data using nested Python dictionaries and lists.
- **Terminal UI / UX:** Mastered the PrettyTable package for CLI visualization, dynamic header creation, and cell border customization.

14. Future Enhancements

- **Persistent Storage:** Integrating SQLite or PostgreSQL database persistence using an ORM such as SQLAlchemy.
- **GUI / Web Portal:** Transitioning the terminal interface to a modern web application using Flask/FastAPI or a desktop GUI with Tkinter/CustomTkinter.

15. References

- **1.** Python Software Foundation. Python 3 Documentation (Data Structures & Dictionaries).
- **2.** PrettyTable Package Documentation (PyPI).
- **3.** Project Source Code: bookstore_management.py.